4 РАЗРАБОТКА ПРОГРАММНЫХ МОДУЛЕЙ

В данном разделе приведено описание ключевых фрагментов кода программных модулей разрабатываемого приложения, а также рассмотрены основные алгоритмы, реализующие его функциональность. Основной акцент сделан на элементах программы, которые обеспечивают обработку данных, работу с пользователями и выполнение основных функций системы. Изучение данных фрагментов даёт представление об общей структуре приложения и логике работы его компонентов.

Схема программы представлена на чертеже ГУИР.400201.311 ПД.

4.1 Представления

Так как в мобильном приложении используется большое количество экранов и вспомогательных компонентов, для описания выбраны представления, которые являются наиболее значимыми для основных сценариев работы пользователя.

4.1.1 Представление создания мероприятия

Представление `EventFormView` используется для создания волонтёрского мероприятия. В нём пользователь указывает название, направление, описание, место проведения, дату и время начала, дату и время окончания, количество волонтёров и фотографию мероприятия. Для хранения состояния формы используется объект `EventViewModel`, который создаётся как `StateObject`. Если мероприятие создаёт подтверждённая организация, дополнительно отображается поле выбора количества рейтинговых очков за участие.

Фрагмент исходного кода представления:

```
struct EventFormView: View {
    @StateObject private var viewModel: EventViewModel
    @State private var isConfirmPresented = false
    private func proceedToConfirmation() {
        guard viewModel.canSubmit else {
            viewModel.errorMessage = "Заполните все
обязательные поля"
            return
        }
        isConfirmPresented = true
    }
}
```

В данном фрагменте показана основная структура формы: экран хранит состояние модели представления `EventViewModel`, открывает выбор местоположения и переводит пользователя к подтверждению только после проверки обязательных полей.

Проверка выполняется через вычисляемое свойство `canSubmit`, а при ошибке пользователь получает сообщение в интерфейсе.

4.1.2 Компонент карточки мероприятия

Компонент `EventSummaryCard` используется для отображения информации о мероприятии. Он применяется в списке мероприятий, на экране мероприятий пользователя и в представлении подробной информации о событии. Компонент принимает объект `EventResponse` и на его основе формирует карточку с названием, статусом, направлением, описанием, местом проведения, датой, временем и количеством требуемых волонтёров.

Фрагмент исходного кода компонента:

```
struct EventSummaryCard: View {
    let event: EventResponse
    var showFullDescription: Bool = false
    var body: some View {
        VStack(alignment: .leading, spacing: 0) {
            eventThumbnail
            Text(event.title).lineLimit(2)
            Text(event.description)
                .lineLimit(showFullDescription ? nil : 3)
        }
    }
}
```

В карточке мероприятия используются данные `EventResponse`: в приведённом фрагменте показан вывод изображения, заголовка и описания. `eventThumbnail` отвечает за отображение изображения мероприятия в карточке: при наличии ссылки на изображение загружает фото через `AsyncImage`, а при отсутствии изображения выводит стандартную заглушку.

Данный компонент является повторно используемым. При отображении в списке описание ограничивается тремя строками, а при открытии подробной карточки параметр `showFullDescription` позволяет вывести полное описание.

Дополнительно карточка выводит значок рейтингового события, если мероприятие создано подтверждённой организацией и сервер вернул количество очков `rating_points`.

4.1.3 Представление выбора местоположения на карте

Для выбора местоположения используется представление `EventMapView`. Так как в SwiftUI отсутствует полный аналог `MKMapView` с требуемой логикой обработки нажатий и изменения области карты, компонент реализован через `UIViewRepresentable`.

Фрагмент исходного кода представления:

```
struct EventMapView: UIViewRepresentable {
```

```
        let cameraCoordinate: CLLocationCoordinate2D
        let onTap: (CLLocationCoordinate2D) -> Void
        func makeUIView(context: Context) -> MKMapView {
            let mapView = MKMapView(frame: .zero)
            mapView.delegate = context.coordinator
            mapView.isRotateEnabled = false
            mapView.addGestureRecognizer(UITapGestureRecognizer(
                target: context.coordinator, action:
#selector(Coordinator.handleTap(_:))))
            return mapView
        }
    }
```

В методе `makeUIView` создаётся объект `MKMapView`, который используется для отображения интерактивной карты внутри SwiftUI-представления. Для карты назначается координатор `context.coordinator`, через который обрабатываются события компонента. Далее задаются основные параметры отображения: включается масштабирование и прокрутка карты, отключается поворот, а также устанавливается начальная область просмотра с центром в координате `cameraCoordinate`. После этого к карте добавляется обработчик нажатия `UITapGestureRecognizer`. При касании карты координаты выбранной пользователем точки определяются в координаторе и передаются в модель представления через замыкание `onTap()`.

4.1.4 Представление списка уведомлений

Представление `NotificationsListView` используется для отображения списка уведомлений пользователя. Данные для списка хранятся в `NotificationViewModel`, который загружает уведомления с сервера и отслеживает состояние загрузки, ошибки и пустого списка. Если уведомления отсутствуют, пользователю показывается соответствующее сообщение, а при наличии данных формируется прокручиваемый список.

Каждое уведомление отображается отдельной строкой с именем отправителя, текстом сообщения и датой создания. Для перехода к подробной информации используется `NavigationLink`, который открывает экран `NotificationView`. Одновременно с переходом выполняется асинхронная операция `markAsRead`, поэтому выбранное уведомление помечается как прочитанное без дополнительного действия со стороны пользователя.

Если уведомление связано с заявкой на участие в мероприятии и заявка находится в состоянии `pending`, детальный экран предоставляет переход к управлению заявками. В этом случае открывается `EventParticipantsManagementView` с выбранным разделом заявок, где организатор может принять или отклонить участника. Таким образом, представление уведомлений не только выводит сообщения пользователю, но и связывает их с дальнейшими действиями в приложении.

Фрагмент исходного кода представления:

```
    ForEach(viewModel.notifications) { item in
        NavigationLink {
            NotificationView(notification:    item,    session:
session)
        } label: {
            notificationRow(item)
        }
        .simultaneousGesture(TapGesture().onEnded {
            Task { await viewModel.markAsRead(item) }
        })
    }
```

4.1.5 Представление оценки участников

После завершения мероприятия приложение переводит событие в фазу оценки участников. Экран `EventRatingView` загружает список профилей, доступных для оценивания, через метод `fetchRatableProfiles()`. Для каждого участника отображается имя, аватар, текущий рейтинг и набор вариантов оценки. Пользователь может выбрать отрицательную, нейтральную или положительную оценку, а для мероприятий от подтверждённых организаций дополнительно доступна оценка звёздочкой, связанная с количеством очков события.

При отправке оценок метод `submitRatings()` формирует массив объектов `RatingItem`. Для каждого профиля берётся выбранное пользователем значение из словаря `selectedScores`. Если оценка не была выбрана, используется значение 0, то есть нейтральная оценка. Далее рассчитывается итоговое количество очков, которое будет передано на сервер.

Отдельно обрабатывается случай выбора рейтинговой звезды. Если мероприятие создано подтверждённой организацией и выбранная оценка совпадает с количеством очков события, к базовой оценке добавляется бонус от текущего рейтинга участника. После расчёта создаётся объект `RatingItem`, содержащий идентификатор оцениваемого профиля и итоговое значение оценки.

Сформированный массив оценок отправляется на сервер через авторизованный запрос `submitRatings()`. После успешного выполнения сервер возвращает обновлённую модель мероприятия, которая передаётся в обработчик `onCompleted()`. Это позволяет закрыть экран оценки и обновить состояние события в интерфейсе. Если при отправке возникает ошибка, её текст сохраняется в `errorMessage` и отображается пользователю.

Фрагмент исходного кода отправки оценок:

```
    let items = profiles.map { profile -> RatingItem in
        let raw: Int = (selectedScores[profile.profileID] ?? nil)
?? 0
        let score: Int
        if raw == eventRatingPoints && isOrganizerVerified {
            let bonusPct = profile.isLeader ? 0.30 : 0.20
```

```
            let bonus = Int(Double(profile.currentRating) *
bonusPct)
            score = raw + bonus
        } else {
            score = raw
        }
        return RatingItem(profileID: profile.profileID, score:
score)
    let updated = try await session.performAuthorizedRequest {
token in
        try await api.submitRatings(eventID: eventID, ratings:
items, token: token)
    }
    onCompleted?(updated)
```

4.2 Алгоритмы реализации основных сценариев

Некоторые алгоритмы программного средства в исходном коде представлены не одним методом, а несколькими взаимосвязанными функциями. Это связано с тем, что отдельные процессы приложения являются достаточно объёмными и включают несколько последовательных этапов. Для повышения читаемости кода и упрощения его дальнейшего сопровождения крупные действия были разделены на более мелкие операции: проверку введённых данных, формирование запроса, отправку данных, обработку полученного результата и изменение состояния интерфейса. Такой подход делает структуру программы более понятной, а также облегчает поиск ошибок и внесение изменений в отдельные части приложения. Ниже приведены алгоритмы основных реализаций проекта, которые отражают логику работы ключевых функций разрабатываемого программного средства.

4.2.1 Алгоритм создания мероприятия

Целью алгоритма создания мероприятия является обработка сервером данных, полученных от мобильного приложения, и подготовка новой записи мероприятия к сохранению в базе данных. На сервере данный сценарий выполняется после POST-запроса по маршруту `/api/events`. Запрос доступен только авторизованному пользователю, поэтому перед созданием мероприятия сервер получает текущего пользователя на основании токена авторизации.

Перед выполнением основной логики данные запроса проверяются через `CreateEventRequest`. В ней описаны обязательные поля мероприятия: название, направление, описание, страна, город, адрес, координаты, дата начала, дата окончания, количество волонтёров и количество рейтинговых очков. Также проверяются ограничения: координаты должны находиться в допустимых пределах, количество волонтёров должно быть не меньше одного, а дата окончания не должна быть раньше даты начала.

После проверки запроса управление передаётся в метод `create_event` сервиса `EventService`. Сначала проверяется, активна ли

учётная запись пользователя. Если пользователь заблокирован, выполнение алгоритма прекращается и возвращается ошибка доступа. Затем сервер получает профиль создателя мероприятия и находит направление события по названию, переданному в запросе.

Фрагмент исходного кода алгоритма:

```
    def create_event(self, db: Session, payload: CreateEventRequest, user: User, background_tasks: BackgroundTasks | None = None) -> EventResponse:
        if not user.is_active:
            raise HTTPException(status_code=status.HTTP_403_FORBIDDEN,
                                detail="User is blocked")
        creator_profile = self._profile_for_user(db, user)
        direction = self._direction_by_name(db, payload.direction)
        is_verified_org = (creator_profile.type == ProfileType.organization
            and bool(creator_profile.is_verified))
        rating_points = 50
        if is_verified_org and payload.rating_points is not None:
            rating_points = max(50, min(100, payload.rating_points))
```

В данном фрагменте показан начальный этап создания мероприятия на сервере. Метод получает объект базы данных, проверенные данные запроса, текущего пользователя и объект фоновых задач. Проверка `user.is_active` необходима для запрета создания мероприятий заблокированными пользователями. После этого сервер получает профиль пользователя и определяет направление мероприятия.

Шаги алгоритма:

Шаг 1. Начало.

Шаг 2. Принять POST-запрос на создание мероприятия по маршруту `/api/events`.

Шаг 3. Получить текущего пользователя по токену авторизации.

Шаг 4. Проверить данные запроса с помощью `CreateEventRequest`.

Шаг 5. Проверить активность учётной записи пользователя.

Шаг 6. Получить профиль пользователя, создающего мероприятие.

Шаг 7. Найти направление мероприятия по названию из запроса.

Шаг 8. Определить, является ли создатель подтверждённой организацией.

Шаг 9. Рассчитать количество рейтинговых очков мероприятия.

Шаг 10. Создать объект `Event` со статусом `pending`.

Шаг 11. Привязать к мероприятию выбранное направление.

Шаг 12. Если создатель является волонтёром, создать для него заявку со статусом `accepted`.

Шаг 13. Создать уведомление о том, что событие отправлено на

подтверждение.

Шаг 14. Сохранить изменения в базе данных.

Шаг 15. Сформировать и вернуть объект `EventResponse`.

Шаг 16. Конец.

4.2.2 Алгоритм извлечения контекста местоположения профиля

Данный алгоритм используется при создании мероприятия. Его цель заключается в извлечении страны, города и имени организатора из модели `ProfileResponse`, полученной с сервера. На основании этих данных формируется контекст местоположения `EventLocationContext`, который затем используется при поиске адреса и выборе точки на карте.

Фрагмент исходного кода алгоритма:

```
    let profile = try await session.performAuthorizedRequest { token in
        try await profileAPI.getMyProfile(token: token)
    }
    let country = canonicalCountry(from: profile.country)
    let city = canonicalCity(from: profile.city, in: country)
    organizerName = displayName(from: profile)
    isOrganizerVerified = profile.isVerified ?? false
    isVerifiedOrganization = (profile.type == "organization") && (profile.isVerified ?? false)
    let searchArea = CityDirectory.searchArea(for: country)
    let cityCoordinate = try await geocodingAPI.geocodeCity(city: city, country: country, area: searchArea)
        ?? searchArea.center
```

Шаги алгоритма:

Шаг 1. Начало.

Шаг 2. Проверить, не был ли контекст местоположения уже загружен ранее.

Шаг 3. Установить флаг `isProfileContextLoading` в значение `true`.

Шаг 4. Выполнить авторизованный запрос `getMyProfile` и получить модель `ProfileResponse`.

Шаг 5. Привести страну и город пользователя к значениям, поддерживаемым справочником городов.

Шаг 6. Сформировать имя организатора и определить, является ли профиль подтверждённой организацией.

Шаг 7. Получить область поиска для страны и координаты города.

Шаг 8. Сформировать `EventLocationContext` для экрана выбора адреса.

Шаг 9. Если запрос профиля или геокодирование завершились ошибкой, сформировать контекст по умолчанию.

Шаг 10. Конец.

4.2.3 Алгоритм выбора местоположения

Целью алгоритма выбора местоположения является определение адреса по координатам, выбранным пользователем на карте. Алгоритм реализован в `EventLocationPickerViewModel`, где хранится выбранная локация, координаты карты, строка поиска и сообщения об ошибках.

При изменении положения карты модель представления получает новые координаты и проверяет, нужно ли выполнять повторное обратное геокодирование. Для этого метод `shouldReverseGeocode(for: coordinate)` сравнивает новую точку с последней обработанной координатой. Если расстояние между ними меньше установленного порога, запрос не отправляется.

Если координаты изменились значительно, предыдущая задача отменяется, после чего создаётся новая асинхронная задача с задержкой 450 миллисекунд. Это уменьшает количество запросов при перемещении карты. После задержки вызывается `reverseGeocode(for: coordinate)`, который получает адрес выбранной точки и сохраняет его в `selectedLocation`.

Фрагмент исходного кода алгоритма:

```
guard shouldReverseGeocode(for: coordinate) else { return }
reverseTask?.cancel()
reverseTask = Task { [weak self] in
    do {
        try await Task.sleep(nanoseconds: 450_000_000)
    } catch {
        return
    }
    await self?.reverseGeocode(for: coordinate)
}
```

Шаги алгоритма:

Шаг 1. Начало.

Шаг 2. Получить координаты центра карты после изменения видимой области.

Шаг 3. Если это первое автоматическое изменение области карты, не выполнять обратное геокодирование.

Шаг 4. Проверить необходимость повторного определения адреса с помощью `shouldReverseGeocode()`.

Шаг 5. Если расстояние между последней обработанной точкой и новой точкой меньше порогового значения, завершить алгоритм.

Шаг 6. Отменить ранее запущенную задачу обратного геокодирования.

Шаг 7. Создать новую асинхронную задачу с задержкой 450 миллисекунд.

Шаг 8. После задержки вызвать `reverseGeocode()` для выбранных координат.

Шаг 9. Если адрес найден, сохранить его в `selectedLocation` и

обновить строку поиска.

Шаг 10. Если адрес не найден, вывести сообщение пользователю.

Шаг 11. Конец.

4.2.4 Алгоритм обработки заявки на участие в мероприятии

Целью алгоритма является обработка заявки пользователя на участие в мероприятии, а также изменение её статуса организатором. На сервере данный сценарий включает два основных действия: отправку заявки волонтёром и её рассмотрение создателем мероприятия.

При отправке заявки сервер сначала получает текущего пользователя по токену авторизации и проверяет активность его учётной записи. Затем по идентификатору находится мероприятие, определяется профиль пользователя и проверяется, существует ли уже заявка данного волонтёра на это мероприятие. Если заявка уже была отправлена, принята или отклонена, сервер возвращает соответствующее сообщение и не создаёт новую запись.

Если повторной заявки нет, сервер проверяет количество свободных мест. Для этого сравнивается количество уже принятых участников с полем `volunteers_needed` у мероприятия. Если свободных мест не осталось, возвращается ошибка. В противном случае создаётся объект `EventApplication` со статусом `pending`, после чего организатору мероприятия отправляется уведомление о новой заявке.

Рассмотрение заявки выполняется отдельным методом `_review_application`. Он используется как для принятия, так и для отклонения заявки. Сначала сервер получает заявку по идентификатору и проверяет, существует ли она. Затем проверяется, является ли текущий пользователь создателем мероприятия. Это необходимо для того, чтобы только организатор мог принимать или отклонять заявки участников.

Фрагмент исходного кода обработки заявки:

```
    def _review_application(
        self, db: Session,*, application_id: int, user: User,
        new_status: EventApplicationStatus,
    ) -> EventResponse:
        application = db.get(EventApplication, application_id)
        if application is None:
            raise
HTTPException(status_code=status.HTTP_404_NOT_FOUND,
                                  detail="Application not found")
        event = application.event
        if event.creator_id != user.id:
            raise
HTTPException(status_code=status.HTTP_403_FORBIDDEN,
                                  detail="Only organizer can
review application")
        if new_status == EventApplicationStatus.accepted:
            accepted_count = self._accepted_count(db, event.id)
            if application.status !=
```

```
EventApplicationStatus.accepted and \
                accepted_count >= event.volunteers_needed:
                 raise
HTTPException(status_code=status.HTTP_409_CONFLICT,
                                        detail="Свободных мест
больше нет")
```

В данном фрагменте показана основная проверочная часть алгоритма рассмотрения заявки. Сервер получает заявку из базы данных, проверяет её наличие и убеждается, что текущий пользователь действительно является организатором соответствующего мероприятия. Если заявка принимается, дополнительно проверяется наличие свободных мест.

Шаги алгоритма обработки заявки:

Шаг 1. Начало.

Шаг 2. Получить текущего пользователя по токену авторизации.

Шаг 3. При подаче заявки найти мероприятие по идентификатору.

Шаг 4. Получить профиль пользователя, который отправляет заявку.

Шаг 5. Проверить, существует ли уже заявка этого пользователя на мероприятие.

Шаг 6. Проверить наличие свободных мест.

Шаг 7. Если мест достаточно, создать заявку со статусом `pending`.

Шаг 8. Отправить организатору уведомление о новой заявке.

Шаг 9. При рассмотрении заявки получить её по идентификатору.

Шаг 10. Проверить, что текущий пользователь является организатором мероприятия.

Шаг 11. Если заявка принимается, повторно проверить количество свободных мест.

Шаг 12. Изменить статус заявки на `accepted` или `rejected`.

Шаг 13. Отправить волонтёру уведомление о результате рассмотрения.

Шаг 14. Сохранить изменения в базе данных.

Шаг 15. Вернуть обновлённую модель мероприятия.

Шаг 16. Конец.

4.2.5 Алгоритм подтверждения присутствия и группировки

Целью алгоритма является фиксация фактического присутствия участников после начала мероприятия и подготовка события к этапу группировки. На сервере данный сценарий выполняется после запроса по маршруту `/api/events/{event_id}/attendance/confirm`. Клиент передаёт список идентификаторов заявок тех участников, которых организатор отметил как присутствующих.

Сервер преобразует список идентификаторов присутствующих заявок в множество `present_set`. Затем из базы данных загружаются все записи посещаемости, связанные с мероприятием. Для каждой записи сервер устанавливает флаг `is_present`: если идентификатор заявки есть в `present_set`, участник считается присутствующим, иначе он отмечается

как отсутствующий. Также фиксируется время проверки в поле checked_at.

Если участник отсутствует, сервер уменьшает его рейтинг на 50 баллов. Это позволяет учитывать неявку на мероприятие в общей рейтинговой системе. После обработки всех записей сервер подсчитывает количество присутствующих участников и сохраняет его в поле present_count.

Фрагмент исходного кода подтверждения присутствия:

```
    present_set = set(present_application_ids)
    attendance_records = db.scalars(
        select(EventAttendance).where(EventAttendance.event_id
== event_id)).all()
    now = datetime.now(UTC)
    for record in attendance_records:
        record.is_present = record.application_id in present_set
        record.checked_at = now
        if record.application_id not in present_set:
            application = db.get(EventApplication,
record.application_id)
            if application is not None:
                self._adjust_rating(
                    db, application.volunteer, -50,
                    event_id=event_id, event_title=event.title,)
```

В данном фрагменте отражена основная часть обработки присутствия. Сервер получает все записи посещаемости мероприятия и сравнивает их с набором заявок, переданным клиентом. Если участник отсутствует, его рейтинг уменьшается с помощью метода _adjust_rating. После завершения цикла сервер сохраняет общее количество присутствующих участников.

Шаги алгоритма:

Шаг 1. Начало.

Шаг 2. Получить мероприятие по идентификатору.

Шаг 3. Проверить, что текущий пользователь является организатором мероприятия.

Шаг 4. Проверить, что мероприятие находится в статусе attendance.

Шаг 5. Получить список идентификаторов заявок присутствующих участников.

Шаг 6. Загрузить из базы данных все записи посещаемости мероприятия.

Шаг 7. Для каждой записи установить признак присутствия is_present.

Шаг 8. Для отсутствующих участников уменьшить рейтинг на 50 баллов.

Шаг 9. Подсчитать количество присутствующих участников.

Шаг 10. Если присутствующих два или больше, перевести мероприятие в статус grouping.

Шаг 11. Если присутствующих меньше двух, перевести мероприятие в статус active.

Шаг 12. При переходе в grouping выбрать количество групп.

Шаг 13. Создать недостающие группы или удалить лишние.

Шаг 14. Распределить присутствующих участников по группам.

Шаг 15. Проверить, что в каждой группе назначен лидер.

Шаг 16. Подтвердить группировку и перевести мероприятие в статус `active`.

Шаг 17. Конец.

На стороне клиента перед подтверждением загружается список участников, после чего формируется набор идентификаторов присутствующих заявок. Этот набор передаётся на сервер в запросе подтверждения присутствия:

```
    let items = try await session.performAuthorizedRequest { token in
        try await api.fetchAttendance(eventID: eventID, token: token)}
    attendanceItems = items
    let serverPresent = Set(items.filter { $0.isPresent }.map { $0.applicationID })
    presentIDs = serverPresent.isEmpty ? Set(items.map { $0.applicationID }) : serverPresent
```

4.2.6 Алгоритм начисления рейтинга участникам

Целью алгоритма является сохранение оценок участников мероприятия и изменение их рейтинга на сервере. Данный сценарий выполняется после получения POST-запроса по маршруту `/api/events/{event_id}/ratings`. Клиент передаёт массив оценок, где для каждого участника указан идентификатор профиля и количество начисляемых или снимаемых баллов.

Сначала сервер находит мероприятие по идентификатору. Если мероприятие отсутствует, возвращается ошибка. Затем проверяется текущий статус события. Начисление рейтинга допускается только в фазе `rating`, так как оценки должны отправляться после завершения основной части мероприятия и подтверждения присутствия участников.

В фрагменте создаётся запись EventRating, которая фиксирует оценку участника в рамках конкретного мероприятия. В запись сохраняются идентификатор события, профиль пользователя, отправившего оценку, профиль оцениваемого участника и количество начисляемых баллов. После добавления записи вызывается метод `_adjust_rating`, который изменяет текущий рейтинг профиля и сохраняет историю изменения рейтинга.

Фрагмент исходного кода отправки оценок:

```
    rating_record = EventRating(
        event_id=event_id,
        rater_id=profile.id,
        rated_id=item.profile_id,
        score=item.score,
    )
```

```
db.add(rating_record)
self._adjust_rating(
    db,
    rated_profile,
    item.score,
    event_id=event_id,
    event_title=event.title,)
```

Шаги алгоритма:

Шаг 1. Начало.

Шаг 2. Получить мероприятие по идентификатору.

Шаг 3. Проверить, что мероприятие находится в статусе `rating`.

Шаг 4. Получить профиль пользователя, который отправляет оценки.

Шаг 5. Для каждой оценки проверить диапазон баллов от −50 до 200.

Шаг 6. Получить профиль оцениваемого участника.

Шаг 7. Проверить, не существует ли уже оценка этого участника.

Шаг 8. Создать запись `EventRating`.

Шаг 9. Изменить рейтинг оцениваемого профиля на значение оценки.

Шаг 10. Создать запись в истории рейтинга `RatingHistory`.

Шаг 11. Если оценки отправил организатор, перевести мероприятие в статус `completed`.

Шаг 12. Сохранить изменения в базе данных.

Шаг 13. Вернуть обновлённую модель мероприятия.

Шаг 14. Конец.

4.3 Выводы по разделу

В данном разделе были рассмотрены основные программные модули разработанного мобильного приложения для организации волонтёрской деятельности. Описаны ключевые представления, а также приведены основные алгоритмы, реализующие функциональность приложения. Рассмотренные фрагменты кода показывают взаимодействие клиентской части приложения с серверной частью, обработку пользовательских данных и обновление состояния интерфейса.

Таким образом, разработанные программные модули формируют основу для дальнейшего тестирования программного средства.